Basic Git Commands Reminder

1. **`git add`** - Stages changes in your working directory for the next commit

```
git add <filename>  # Add a specific file
git add .           # Add all changes in the current directory
```

---

2. **`git commit`** - Saves your staged changes to the local repository with a message

```
git commit -m "the commit message"
```

---

3. **`git pull`** - Fetches changes from a remote repository and merges them into your local branch.

```
git pull origin main   # Pull changes from the 'main' branch
```

---

4. **`git push`** - Sends your committed changes to a remote repository

```
git push origin main   # Push local commits to the 'main' branch
```

---

5. **`git stash`** - Temporarily saves your uncommitted changes so you can work on something else

```
git stash              # Stash current changes
```

---

6. **`git stash pop -`** Applies the most recently stashed changes and removes them from the stash list.

```
git stash pop          # Apply last stash
```

---

7. **`git status -`** Shows the current state of your working directory and staging area. It tells you which files are staged for commit, which are modified but unstaged, and which are untracked

Commit vs Commit --amend

- **`git commit`** - Creates a new commit with staged changes. Use it for new, separate snapshot.
- **`git commit --amend -`** Edits the most recent commit to fix mistakes or update the message without adding a new commit.

Why use `commit --amend`:

- Keeps history clean by fixing the last commit instead of adding extra ones.
- Makes rebasing easier by maintaining a tidy commit history (you'll learn about rebasing later).

When to use a normal commit:

- For new, distinct changes.
- If the commit is already pushed and shared (unless you force push carefully).

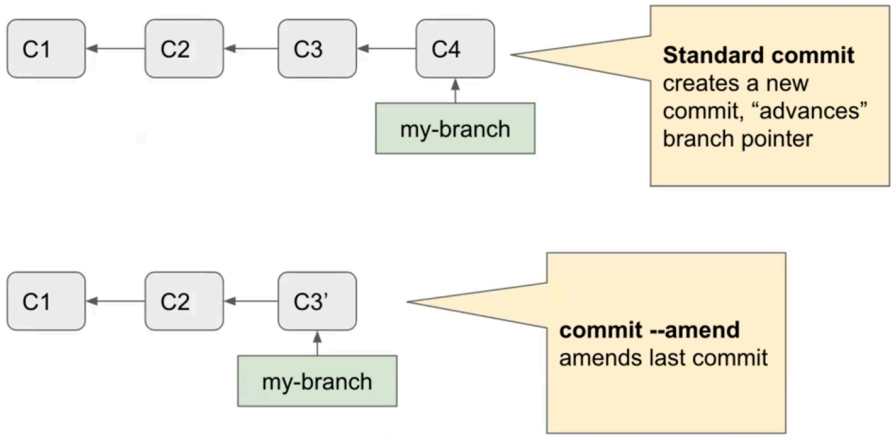
Important: After amending a commit that's been pushed, you must force push:

```
git push -f origin <branch>
```

Force push rewrites history, so use it carefully.

Example:

```
git commit -m "Add feature"
git add missed_file.js
git commit --amend
git push -f origin main
```



Branches

- It's best practice to work on **small, focused branches** instead of big, long-lived ones. This makes your changes easier to review, test, and merge.
- Regularly **rebase** or **merge** your branch with `main` to stay up to date and avoid big conflicts.

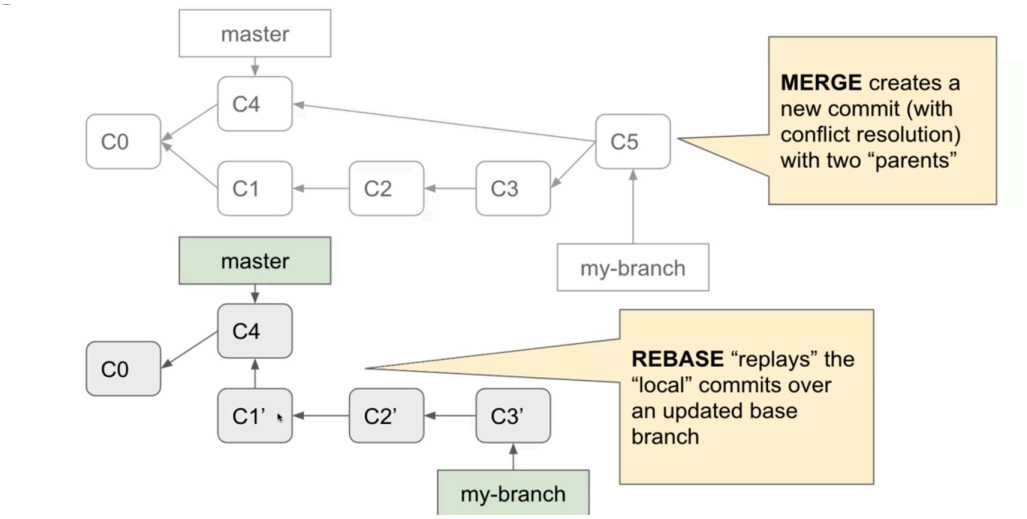
Basic branch commands:

```
git branch              # List all local branches
git branch <name>       # Create a new branch
git checkout <name>     # Switch to a branch
git checkout -b <name> # Create and switch to a new branch
```

Merge vs Rebase

Both **merge** and **rebase** are ways to integrate changes from one branch into another, but they work differently and have different effects on your project history.

Aspect	**Merge**	**Rebase**
History	Combines histories by creating a new merge commit, preserving branch history	Moves your commits on top of another branch, creating a linear history
Commit graph	Branching structure is preserved	History looks like a straight line
Conflict handling	Conflicts are resolved once during merge commit	Conflicts may occur at each commit being replayed
Use case	When you want to keep the full history and context	When you want a clean, linear commit history



How to Use Merge:

```
git checkout -b feature-branch  # Create and switch to new branch
git add .
git commit -m "My feature work" # Work and commit changes
git pull origin main            #Update local feature-branch with
                                 latest changes
```

- ✅ If there are **no conflicts**, simply:
  `git push origin main`
- ❌If there are conflicts, your IDE (like IntelliJ IDEA) will mark them.
  - Resolve conflicts in IDEA
  - Then:
    ```
    git add .
    git commit                          # Finish the merge
    git push origin feature-branch
    ```

How to Use Rebase:

```
git checkout -b feature-branch  # Create and switch to new branch
git add .
git commit -m "My feature work" # Work and commit changes

# 💡 While working, prefer using:
git commit --amend              # To keep history clean and make
                                 rebasing easier

git pull --rebase origin main   # 3. Rebase your local main on top
                                   of the latest from remote
```

- ✅ If there are **no conflicts**, you're done!
- ❌If there **are conflicts**:
  - Resolve them in your IDE (e.g. IntelliJ IDEA)
  - Then:
    ```
    git status          # Check what still needs to be staged
    git add .           # If needed, stage the resolved files
    git rebase --continue
    ```
- 🔁 If your branch has **multiple commits**, you may need to resolve conflicts for **each commit** during the rebase process.

- After rebase is complete:
  `git push origin feature-branch`
- **Note**: If the branch was already pushed before the rebase:
  `git push -f origin feature-branch`

Squash Commits (Before Rebase)

If your branch has many small or messy commits, it's good practice to squash them into one clear commit before rebasing. This keeps history clean and avoids unnecessary conflicts.

```
git rebase -i HEAD~N      # Replace N with the number of recent
                            commits you want to squash
```

- In the interactive rebase menu, change pick to squash (or s) for all commits you want to merge into the one above.
- Save and write a clean commit message.

Now your branch history is clean and ready for rebase.

✅ Best Practices

- 🔄 **Use `git commit --amend` when possible** to avoid unnecessary commits and keep history clean.
- 📥 **Always use rebase** to keep a linear and understandable project history.
- 🕒 **Rebase often** — ideally **at least once a day** — to reduce conflict risk and stay updated.
- 🧹 **Squash commits before (or during) rebase** to keep the branch history concise and easier to manage.